UNIVERSIDAD POPULAR DEL CESAR | PATRONES DE DISEÑO DE SOFTWARE

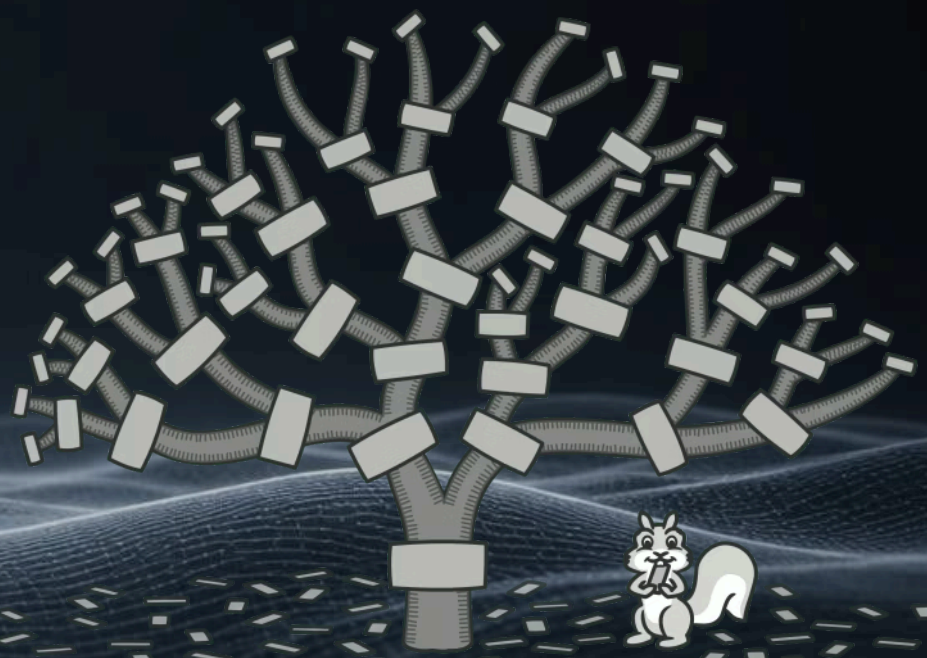
Patrón de Diseño Estructural Composite

Tratamiento uniforme de objetos individuales y jerarquías compuestas

Juan Assia · **Hernando Díaz** · **Juan Guzman** · **David Palomino**

Docente:
Jairo Seoanes

Ingeniería de Sistemas · Octubre de 2026



DEFINICIÓN Y FUNDAMENTOS

¿Qué es el patrón Composite?

Es un patrón de diseño estructural que nos permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

> “Composite permite tratar un objeto individual y una composición de objetos mediante una misma interfaz”.

CATEGORÍA

Patrón estructural

PROPÓSITO

Representar jerarquías parte-todo

CONCEPTO CENTRAL

Tratar de manera uniforme objetos simples y compuestos

MECANISMO

Composición de objetos, polimorfismo y operaciones recursivas

¿Por qué existe Composite?

Si el código cliente trabaja con objetos individuales y grupos que contienen otros objetos; conocer el tipo concreto, distinguir hojas de grupos y recorrer cada nivel a mano complica la lógica y cuesta mantenerla.

Sin Composite

- El cliente distingue entre tipos concretos y necesita condicionales para procesar cada tipo
- Las estructuras anidadas se complican
- Las nuevas clases pueden exigir cambios en el cliente

Con Composite

- Los elementos comparten una interfaz común; las hojas realizan su propia operación
- Los compuestos delegan la operación a sus hijos; la estructura se recorre recursivamente
- El cliente trata elementos simples y compuestos de manera uniforme

Aclaración: Composite no elimina todos los condicionales; reduce la necesidad de distinguir hojas y compuestos en las operaciones de la interfaz. No promete mejor rendimiento: el beneficio es estructural y reduce el acoplamiento del cliente con clases concretas.

¿Cuándo utilizar Composite?

01 **Árbol de objetos:** El modelo se puede representar como un árbol de objetos.

02 **Elementos y contenedores:** Hay elementos individuales y contenedores que agrupan otros elementos.

03 **Operaciones uniformes:** El cliente ejecuta operaciones uniformes sobre elementos simples y compuestos.

04 **Propagación recursiva:** Interesa propagar operaciones recursivamente por toda la estructura.

EDITORES GRÁFICOS

Formas individuales y grupos de formas.

SISTEMAS DE ARCHIVOS

Archivos y directorios anidados.

INTERFACES GRÁFICAS

Paneles con botones, etiquetas y otros paneles.

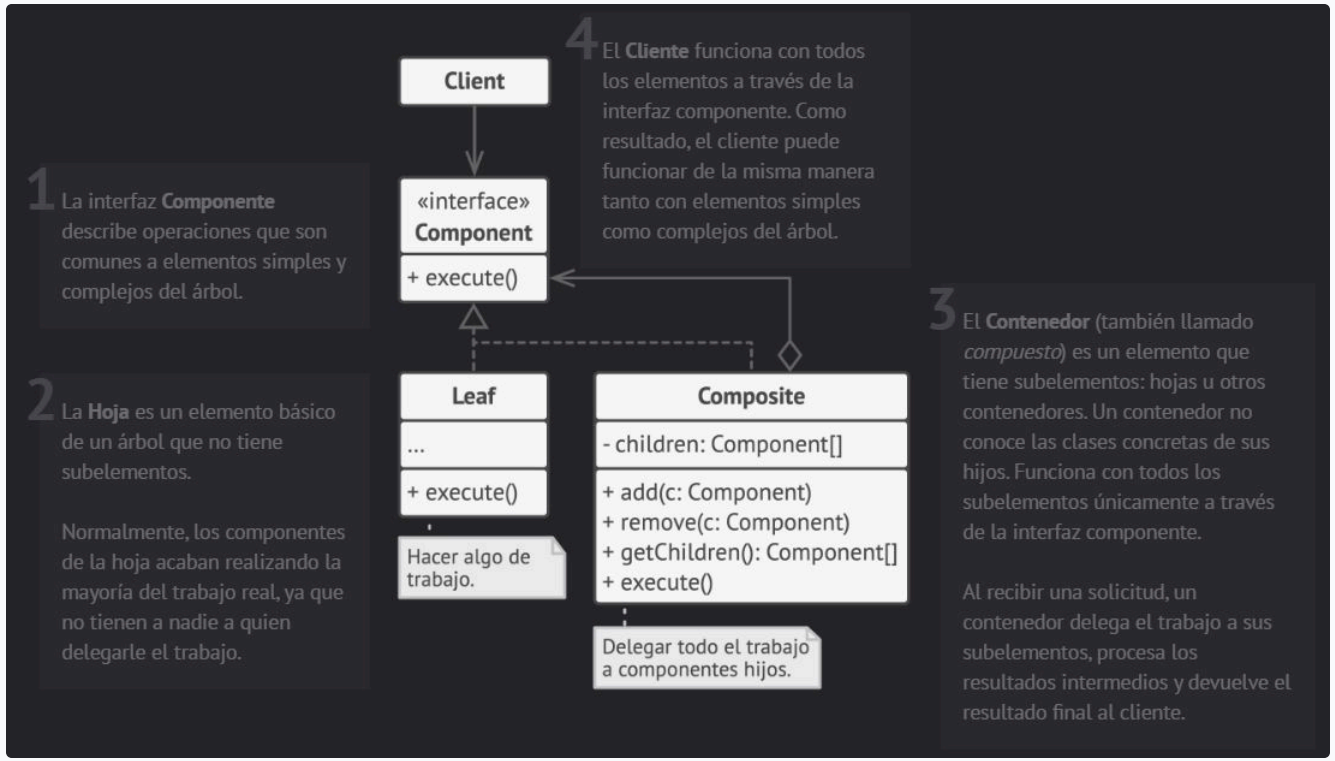
ESTRUCTURAS JERÁRQUICAS

Grupos que contienen grupos y elementos.

▲ Advertencia: No es necesario para cualquier relación entre objetos; resulta útil cuando existe una verdadera jerarquía y se quieren tratar sus niveles de forma uniforme.

ESTRUCTURA DEL PATRÓN

Arquitectura de Composite



COMPONENT

Interfaz común que declara las operaciones compartidas para hojas y compuestos.

LEAF (HOJA)

Elemento individual sin hijos. Ejecuta la operación concreta de forma autónoma.

COMPOSITE (COMPUESTO)

Almacena una colección de `Component` y delega la ejecución recorriendo a sus hijos.

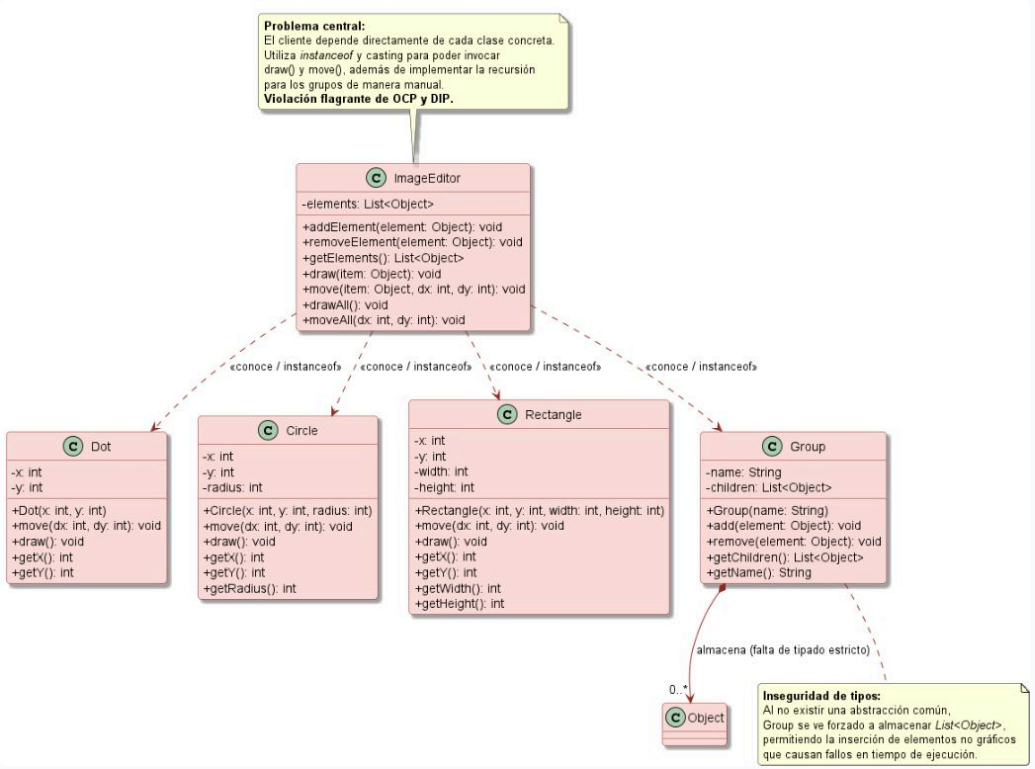
CLIENT

Interactúa con la estructura exclusivamente a través de `Component`.

DISEÑO INICIAL SIN EL PATRÓN

Caso práctico: editor gráfico sin Composite

Un editor gráfico gestiona formas geométricas individuales y agrupaciones. Sin interfaz común, el cliente debe verificar cada tipo en tiempo de ejecución.



- Alto acoplamiento con clases concretas
- Riesgo de error al añadir un nuevo tipo de forma

Código Java sin Composite: inspección manual de tipos

```
void drawElement(Object element) {
    if (element instanceof Dot) {
        ((Dot) element).draw();
    } else if (element instanceof Circle) {
        ((Circle) element).draw();
    } else if (element instanceof Rectangle) {
        ((Rectangle) element).draw();
    } else if (element instanceof Group) {
        Group group = (Group) element;
        for (Object child : group.getChildren()) {
            drawElement(child);
        }
    }
}
```

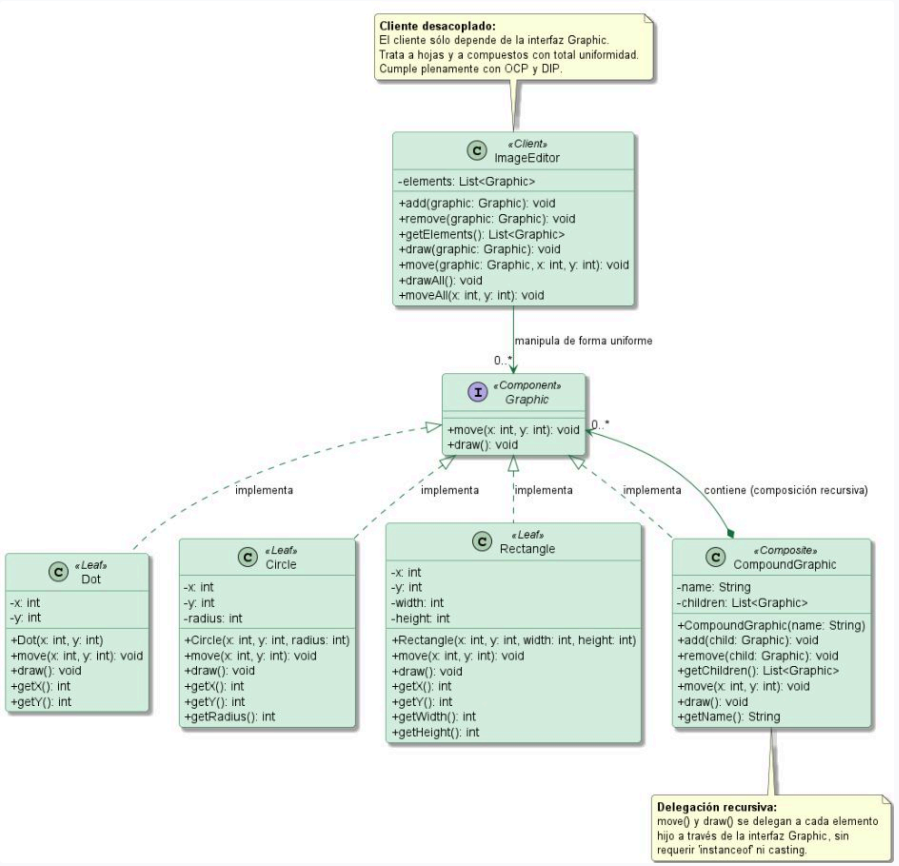
El método `drawElement` asume toda la lógica de deserialización, casteo forzado y recorrido manual de las colecciones de `Group`.

REFACTORIZACIÓN CON EL PATRÓN

Aplicando Composite al editor gráfico

Unificamos hojas y grupos bajo la interfaz `Graphic`. `CompoundGraphic` delega operaciones recursivamente; el cliente llama operaciones sobre `Graphic` sin comprobar el tipo.

UML posterior al patrón



Código Java representativo, consistente con el diagrama:

```
interface Graphic { void move(int x, int y); void draw(); }
class CompoundGraphic implements Graphic {
   private final List<Graphic> children = new ArrayList<>();
   public void add(Graphic child) { children.add(child); }
   public void remove(Graphic child) { children.remove(child); }
   @Override public void move(int x, int y) { for (Graphic child : children) child.move(x, y); }
   @Override public void draw() { for (Graphic child : children) child.draw(); }
}
```

Llamada del cliente: `Graphic g = ...; g.draw();`.

- Interfaz común
- Polimorfismo
- Composición recursiva
- Delegación de operaciones
- Menor dependencia del cliente respecto a clases concretas

¿Qué cambia al aplicar Composite?

ANTES

Formas y grupos sin interfaz común; condicionales para distinguir tipos; el cliente conoce las clases concretas; más complejidad con grupos anidados.

DESPUÉS

Formas y grupos implementan Graphic; el cliente usa la interfaz común; CompoundGraphic delega en sus hijos; los grupos se anidan recursivamente; es más sencillo incorporar nuevas clases que respeten la interfaz.

Comparación de código

Antes: `if (element instanceof Circle) { // Dibujar círculo }`. Después: `Graphic graphic = ...; graphic.draw();` (llamada polimórfica; la creación del objeto puede ocurrir en otra parte del programa).

VENTAJAS

Simplifica árboles de objetos; reduce el acoplamiento del cliente con clases concretas; facilita extender hojas y compuestos compatibles.

LIMITACIÓN

Puede ser innecesariamente complejo si los objetos no forman una jerarquía parte-todo o sus operaciones no se pueden unificar con coherencia.

CIERRE Y RECURSOS

Conclusiones y demostración del código

> “Composite permite construir estructuras jerárquicas y tratar objetos individuales y grupos mediante una interfaz común.”

- Es un patrón estructural. Participantes: Component, Leaf, Composite y Client.
- Usa composición, polimorfismo y recursión; útil en jerarquías con operaciones uniformes.
- Su uso debe justificarse según el problema de diseño.

Muchas gracias.

REPOSITORIO DEL PROYECTO

patron-composite-java

URL:
https://github.com/HernandoDiaz20/patron-composite-java.git

Carpetas:

- before-composite/
- after-composite/

README con propósito, instrucciones de ejecución y diferencias; incluye UML de ambas versiones.